

The Historical Lineage of Runtime Governance Terminology: A Reference Guide for AI Governance Control Planes and Execution-Governance Architectures

John M. Willis
Sustainable Future Tech, Inc.
Gaithersburg, Maryland, USA
Email: jwillis@sustainablefuturetech.com
ORCID: <https://orcid.org/0009-0007-5889-9628>

May 29, 2026

Abstract

As AI governance, agentic systems, runtime authorization architectures, and execution-governance control planes mature, discussions increasingly employ terminology such as state transition, canonical state, commit, binding, authorization, admissibility, invariants, provenance, policy enforcement, Policy Decision Point (PDP), Policy Enforcement Point (PEP), lifecycle state, and distributed ordering.

Because these terms are often encountered by practitioners through contemporary AI governance discussions, there is a risk that their historical origins become obscured. In reality, most of this vocabulary predates modern AI systems by decades, and in several cases by more than half a century.

This article provides a historical survey of the principal concepts frequently used within execution-governance architectures and AI Governance Control Planes (AGCPs). The goal is not to establish ownership of terminology, but rather to document the long-standing academic and engineering traditions from which these concepts emerged.

This paper is intended as a historical and bibliographic reference guide documenting the academic lineage of terminology commonly used in contemporary runtime-governance and execution-governance architectures.

Keywords: AI Governance, Runtime Governance, Execution Governance, Authorization, Distributed Systems, Commit Semantics, Policy Enforcement, Admissibility, State Transitions, Control Planes, Provenance, Formal Methods

Cite as: Willis, J. (2026). The Historical Lineage of Runtime Governance Terminology: A Reference Guide for AI Governance Control Planes and Execution-Governance Architectures (1.0). Zenodo. <https://doi.org/10.5281/zenodo.20448734>

Introduction

Technical fields rarely emerge from entirely new vocabulary.

Instead, new architectural paradigms are typically constructed from existing concepts inherited from mathematics, formal logic, operating systems, transaction processing, distributed systems, computer security, systems engineering, and software architecture.

Runtime governance architectures are no exception.

Many concepts now appearing in discussions of execution governance have deep roots extending across multiple disciplines.

This article surveys those roots and provides representative academic references for each major concept.

Admissibility

The concept of admissibility substantially predates computing.

The term appears throughout:

- mathematics,
- formal logic,
- proof theory,
- optimization,
- control theory,
- and law.

Generally, an admissible action, solution, or proof belongs to the allowable solution space defined by governing constraints.

Representative sources include:

- Kleene (1952)
- Tarski (1956)
- Bellman (1957)

Authorization

Authorization represents one of the foundational concepts of computer security.

Authorization determines whether an actor may perform a specific operation under defined conditions.

Representative sources include:

- Lampson (1974)
- Saltzer and Schroeder (1975)
- Sandhu et al. (1996)

Binding

Binding has roots in formal logic, programming language semantics, compiler design, and type theory.

Binding refers to the association of an abstract reference with a concrete realization.

Examples include variable binding, late binding, dynamic binding, and runtime binding.

Representative sources include:

- Church (1940)
- Scott and Strachey (1971)
- Aho, Sethi, and Ullman (1986)

Canonical State

The notion of a canonical representation predates modern computing and originates in mathematics and formal logic.

Within computing, canonical state generally refers to the authoritative representation of system reality against which evaluation and reasoning are performed.

Distributed systems literature has long relied on authoritative state representations for coordination and consistency.

Representative sources include:

- Knuth (1968)
- Lamport (1978)
- Lynch (1996)

Commit

The concept of commit emerged from transaction-processing research and database systems.

A commit operation transforms a tentative transition into an authoritative and durable state mutation.

This concept forms the foundation of ACID transactions, distributed commit protocols, and modern transaction-processing systems.

Representative sources include:

- Gray (1978)
- Bernstein, Hadzilacos, and Goodman (1987)
- Gray and Reuter (1992)

Commit Boundary

Unlike many terms discussed above, commit boundary does not appear as a widely standardized academic term.

However, it is best understood as a compositional architectural expression derived from multiple established traditions:

- transaction processing,
- commit semantics,
- distributed systems,
- state-transition systems,
- software architecture,
- and execution-control mechanisms.

In this interpretation, a commit boundary represents the architectural point at which a proposed state transition either becomes authoritative and consequential or is prevented from doing so.

The term therefore inherits its semantics from long-established concepts rather than introducing entirely new underlying vocabulary.

Representative antecedents include:

- Gray (1978)
- Bernstein et al. (1987)
- Gray and Reuter (1992)
- Parnas (1972)

Distributed Ordering

Distributed ordering concerns the determination of authoritative event sequencing across distributed participants.

The field emerged directly from distributed systems research.

Representative sources include:

- Lamport (1978)
- Birman and Joseph (1987)
- Lynch (1996)

Invariants

Invariants are central to formal verification, program correctness, and systems engineering.

An invariant is a property that must remain true throughout all valid state transitions.

Representative sources include:

- Hoare (1969)
- Dijkstra (1976)
- Lamport (1977)

Lifecycle State

Lifecycle-state management originates from systems engineering and software lifecycle methodologies.

Lifecycle states represent the stages through which an object, process, request, or resource progresses during its existence.

Representative sources include:

- Boehm (1988)
- ISO/IEC 15288

Policy Decision Points and Policy Enforcement Points

The Policy Decision Point (PDP) and Policy Enforcement Point (PEP) model became formalized within enterprise authorization architectures.

The PDP determines whether an action should be allowed.

The PEP enforces that decision at the execution target.

Representative sources include:

- Yavatkar, et al. (2000)
- OASIS XACML Version 3.0 (2013)

Policy Enforcement

Policy enforcement emerged from security architecture and reference-monitor theory.

The concept concerns the prevention of prohibited actions through enforceable control mechanisms.

Representative sources include:

- Lampson (1974)

- Saltzer and Schroeder (1975)
- Schneider (2000)

Provenance

Provenance has origins in archival science, historical scholarship, and evidentiary recordkeeping.

In computing, provenance became formalized within databases, scientific workflows, and information systems.

Representative sources include:

- Buneman, Khanna, and Tan (2001)
- Moreau et al. (2013)

State Transition

The concept of state transition forms one of the foundational abstractions of computer science.

State-machine theory emerged from early work in automata theory and sequential systems.

A state transition represents movement from one valid system state to another under specified conditions.

Representative sources include:

- McCulloch and Pitts (1943)
- Mealy (1955)
- Moore (1956)
- Hopcroft and Ullman (1979)

State-transition reasoning remains fundamental to workflow engines, distributed systems, formal methods, and runtime governance architectures.

Conclusion

The terminology increasingly used within AI governance, execution-governance architectures, runtime control planes, and AGCP-style systems is deeply rooted in decades of prior academic and engineering work.

While modern architectures may combine these concepts in novel ways, the underlying vocabulary largely represents the continuation and synthesis of long-established traditions spanning formal methods, transaction processing, distributed systems, computer security, systems engineering, and software architecture.

Understanding these historical roots improves conceptual clarity, reduces unnecessary disputes regarding terminology, and helps place emerging governance architectures within their broader intellectual context.

References

- Aho, A. V., Sethi, R., & Ullman, J. D. (1986). *Compilers: Principles, Techniques, and Tools*. Reading, MA: Addison-Wesley.
- Bellman, R. (1957). *Dynamic Programming*. Princeton, NJ: Princeton University Press.
- Bernstein, P. A., Hadzilacos, V., & Goodman, N. (1987). *Concurrency Control and Recovery in Database Systems*. Reading, MA: Addison-Wesley.
- Birman, K. P., & Joseph, T. A. (1987). Reliable communication in the presence of failures. *ACM Transactions on Computer Systems*, 5(1), 47–76.
<https://doi.org/10.1145/7351.7478>
- Boehm, B. W. (1988). A spiral model of software development and enhancement. *Computer*, 21(5), 61–72. <https://doi.org/10.1109/2.59>
- Buneman, P., Khanna, S., & Tan, W.-C. (2001). Why and where: A characterization of data provenance. In *Proceedings of the 8th International Conference on Database Theory (ICDT 2001)* (pp. 316–330). Berlin: Springer. https://doi.org/10.1007/3-540-44503-X_20
- Church, A. (1940). A formulation of the simple theory of types. *Journal of Symbolic Logic*, 5(2), 56–68. <https://doi.org/10.2307/2266170>
- Dijkstra, E. W. (1976). *A Discipline of Programming*. Englewood Cliffs, NJ: Prentice-Hall.
- Gray, J. (1978). Notes on database operating systems. In R. Bayer, R. M. Graham, & G. Seegmüller (Eds.), *Operating Systems: An Advanced Course* (pp. 393–481). Berlin: Springer-Verlag.
- Gray, J., & Reuter, A. (1992). *Transaction Processing: Concepts and Techniques*. San Francisco, CA: Morgan Kaufmann.
- Hoare, C. A. R. (1969). An axiomatic basis for computer programming. *Communications of the ACM*, 12(10), 576–580. <https://doi.org/10.1145/363235.363259>
- Hopcroft, J. E., & Ullman, J. D. (1979). *Introduction to Automata Theory, Languages, and Computation*. Reading, MA: Addison-Wesley.
- ISO/IEC. (2015). *ISO/IEC/IEEE 15288:2015 Systems and Software Engineering — System Life Cycle Processes*. Geneva, Switzerland: International Organization for Standardization.
- Kleene, S. C. (1952). *Introduction to Metamathematics*. Amsterdam: North-Holland Publishing Company.

- Knuth, D. E. (1968). *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Reading, MA: Addison-Wesley.
- Lamport, L. (1977). Proving the correctness of multiprocess programs. *IEEE Transactions on Software Engineering*, SE-3(2), 125–143. <https://doi.org/10.1109/TSE.1977.229904>
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565. <https://doi.org/10.1145/359545.359563>
- Lampson, B. W. (1974). Protection. *ACM SIGOPS Operating Systems Review*, 8(1), 18–24. <https://doi.org/10.1145/775265.775268>
- Lynch, N. A. (1996). *Distributed Algorithms*. San Francisco, CA: Morgan Kaufmann.
- McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(4), 115–133. <https://doi.org/10.1007/BF02478259>
- Mealy, G. H. (1955). A method for synthesizing sequential circuits. *Bell System Technical Journal*, 34(5), 1045–1079. <https://doi.org/10.1002/j.1538-7305.1955.tb03788.x>
- Moore, E. F. (1956). Gedanken-experiments on sequential machines. In C. E. Shannon & J. McCarthy (Eds.), *Automata Studies* (pp. 129–153). Princeton, NJ: Princeton University Press.
- Moreau, L., Missier, P., Belhajjame, K., B'Far, R., Cheney, J., Coppens, S., ... & Tilmes, C. (2013). *PROV-DM: The PROV Data Model*. W3C Recommendation. Available at: <https://www.w3.org/TR/prov-dm/>
- OASIS. (2013). *eXtensible Access Control Markup Language (XACML) Version 3.0*. OASIS Standard. Available at: <https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>
- Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. <https://doi.org/10.1145/361598.361623>
- Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>
- Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *Computer*, 29(2), 38–47. <https://doi.org/10.1109/2.485845>
- Schneider, F. B. (2000). Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1), 30–50. <https://doi.org/10.1145/353323.353382>

Scott, D. S., & Strachey, C. (1971). Toward a mathematical semantics for computer languages. *Oxford Programming Research Group Technical Monograph PRG-6*. Oxford University Computing Laboratory.

Tarski, A. (1956). *Logic, Semantics, Metamathematics: Papers from 1923 to 1938*. Oxford, UK: Oxford University Press.

Yavatkar, R., Pendarakis, D., & Guerin, R. (2000). *A framework for policy-based admission control* (RFC 2753). Internet Engineering Task Force.
<https://doi.org/10.17487/RFC2753>